

# Import Modules and Packages in Python

## 1. What is a Module?

A **module** is a Python file (`.py`) that contains functions, variables, and classes which can be reused in other Python programs.

Example:

Suppose you have a file named `mymath.py`.

```
# mymath.py
def add(a, b):
    return a + b

pi = 3.14
```

You can use this module in another Python file.

---

## 2. Importing a Module

### Syntax

```
import module_name
```

### Example

```
import math

print(math.sqrt(25))
print(math.factorial(5))
```

### Output

```
5.0
120
```

Here, `math` is a built-in Python module.

---

## 3. Import Specific Functions

Instead of importing the entire module, you can import only what you need.

### Syntax

```
from module_name import function_name
```

### Example

```
from math import sqrt  
print(sqrt(36))
```

### Output

```
6.0
```

Multiple imports:

```
from math import sqrt, factorial  
  
print(sqrt(49))  
print(factorial(4))
```

### Output

```
7.0  
24
```

---

## 4. Import Everything Using \*

### Syntax

```
from module_name import *
```

### Example

```
from math import *  
  
print(sqrt(16))  
print(factorial(5))
```

### Output

```
4.0  
120
```

### Note

Using \* is generally **not recommended** because it can create naming conflicts and reduce code readability.

---

## 5. Import with an Alias

An alias is a shorter name given to a module.

### Syntax

```
import module_name as alias
```

### Example

```
import math as m  
print(m.sqrt(64))
```

### Output

8.0

Another example:

```
import numpy as np
```

---

## 6. Import Specific Function with Alias

### Syntax

```
from module_name import function_name as alias
```

### Example

```
from math import factorial as fact  
print(fact(6))
```

### Output

720

---

## 7. Difference Between `import` and `from...import`

| Statement                          | Usage                            |
|------------------------------------|----------------------------------|
| <code>import math</code>           | Use module name before functions |
| <code>from math import sqrt</code> | Use function directly            |
| <code>import math as m</code>      | Use alias                        |
| <code>from math import *</code>    | Imports everything               |

Example:

```
import math
```

```
print(math.sqrt(9))
from math import sqrt
print(sqrt(9))
```

Both give:

3.0

---

## 8. Importing User-Defined Modules

**File: calculator.py**

```
def add(a, b):
    return a + b

def sub(a, b):
    return a - b
```

**File: main.py**

```
import calculator

print(calculator.add(10, 5))
print(calculator.sub(10, 5))
```

### Output

```
15
5
```

---

## 9. What is a Package?

A **package** is a collection of related modules organized in directories.

Example structure:

```
mypackage/
├── __init__.py
├── math_utils.py
└── string_utils.py
```

- math\_utils.py → Module
  - string\_utils.py → Module
  - mypackage → Package
- 

## 10. Importing from a Package

Suppose `math_utils.py` contains:

```
def square(n):  
    return n * n
```

Import it as:

```
from mypackage.math_utils import square  
  
print(square(5))
```

## Output

25

---

## 11. Common Built-in Modules

| Module                   | Purpose                      |
|--------------------------|------------------------------|
| <code>math</code>        | Mathematical functions       |
| <code>random</code>      | Random numbers               |
| <code>datetime</code>    | Date and time                |
| <code>os</code>          | Operating system functions   |
| <code>sys</code>         | Python interpreter functions |
| <code>json</code>        | JSON data handling           |
| <code>statistics</code>  | Statistical calculations     |
| <code>collections</code> | Specialized container types  |

Example:

```
import random  
  
print(random.randint(1, 10))
```

Output (random):

7

---

## 12. Practice Questions

### Easy

1. Import the `math` module and print the square root of 81.
2. Import only `factorial` from `math` and find factorial of 6.
3. Import `random` as `rd` and generate a random integer from 1 to 20.

### Medium

4. Create a module named `greetings.py` containing a function `hello(name)`.
5. Import the module and print a greeting message.
6. Create a package with two modules and import functions from both.

## Advanced

7. Create a calculator module with `add`, `subtract`, `multiply`, and `divide` functions.
  8. Import the module using an alias and perform calculations.
  9. Explain the differences between modules and packages with examples.
- 

## Summary

- **Module:** A single Python file containing reusable code.
- **Package:** A folder containing multiple related modules.
- `import module` → Import the whole module.
- `from module import function` → Import specific items.
- `as` → Create aliases.
- `*` → Import everything (generally avoided).
- Modules and packages help organize code and improve reusability.

**Link :** [CHATGPT](#)

# Common Built-in Modules in Python (Detailed)

Python provides many **built-in modules** that are ready to use without installing anything. These modules help perform common tasks such as mathematical calculations, working with files, generating random numbers, handling dates, and more.

---

## 1. `math` Module

The `math` module provides mathematical functions and constants.

```
import math
```

### Common Functions

#### **`sqrt()` – Square Root**

```
import math

print(math.sqrt(25))
```

#### **Output**

```
5.0
```

---

#### **`factorial()` – Factorial**

```
import math

print(math.factorial(5))
```

#### **Output**

```
120
```

---

#### **`ceil()` – Round Up**

```
import math

print(math.ceil(4.2))
```

#### **Output**

```
5
```

---

### **floor() – Round Down**

```
import math  
print(math.floor(4.9))
```

#### **Output**

```
4
```

---

### **pow() – Power**

```
import math  
print(math.pow(2, 3))
```

#### **Output**

```
8.0
```

---

### **Constants**

```
import math  
  
print(math.pi)  
print(math.e)
```

#### **Output**

```
3.141592653589793  
2.718281828459045
```

---

## **2. random Module**

Used to generate random numbers and select random items.

```
import random
```

#### **randint()**

Returns a random integer.

```
import random  
  
print(random.randint(1, 10))
```

#### **Output**

(Output changes every time.)

---

#### **choice()**

Selects a random element.

```
import random
colors = ["red", "blue", "green"]
print(random.choice(colors))
```

Possible Output:

```
blue
```

---

#### **shuffle()**

Shuffles a list.

```
import random
numbers = [1, 2, 3, 4, 5]
random.shuffle(numbers)
print(numbers)
```

Possible Output:

```
[3, 1, 5, 2, 4]
```

---

#### **random()**

Returns a float between 0 and 1.

```
import random
print(random.random())
```

Possible Output:

```
0.6352
```

---

## **3. datetime Module**

Used to work with dates and times.

```
import datetime
```

## Current Date and Time

```
import datetime  
  
now = datetime.datetime.now()  
  
print(now)
```

Possible Output:

```
2026-06-09 14:30:10.123456
```

---

## Current Date

```
import datetime  
  
today = datetime.date.today()  
  
print(today)
```

Possible Output:

```
2026-06-09
```

---

## Creating a Date

```
import datetime  
  
d = datetime.date(2026, 12, 25)  
  
print(d)
```

**Output**

```
2026-12-25
```

---

## Formatting Dates

```
import datetime  
  
now = datetime.datetime.now()  
  
print(now.strftime("%d-%m-%Y"))
```

Output:

## 4. `os` Module

Used for operating system-related tasks.

```
import os
```

### Current Working Directory

```
import os

print(os.getcwd())
```

Example Output:

```
C:\Users\Krish
```

---

### List Files

```
import os

print(os.listdir())
```

Example Output:

```
['file1.py', 'data.txt']
```

---

### Create Folder

```
import os

os.mkdir("Test")
```

---

### Remove Folder

```
import os

os.rmdir("Test")
```

---

## 5. `sys` Module

Provides information about the Python interpreter.

```
import sys
```

## Python Version

```
import sys
```

```
print(sys.version)
```

Example Output:

```
3.13.0 (...)
```

---

## Command Line Arguments

```
import sys
```

```
print(sys.argv)
```

Example Output:

```
['program.py']
```

---

## Exit Program

```
import sys
```

```
sys.exit()
```

Stops program execution.

---

## 6. json Module

Used to work with JSON data.

```
import json
```

### Python Dictionary to JSON

```
import json
```

```
data = {"name": "Krish", "age": 22}
```

```
json_data = json.dumps(data)
```

```
print(json_data)
```

Output:

```
{"name": "Krish", "age": 22}
```

---

## JSON to Python Dictionary

```
import json

text = '{"name":"Krish","age":22}'

data = json.loads(text)

print(data)
```

Output:

```
{'name': 'Krish', 'age': 22}
```

---

## 7. `statistics` Module

Used for statistical calculations.

```
import statistics
```

### Mean

```
import statistics

nums = [10, 20, 30, 40]

print(statistics.mean(nums))
```

Output:

```
25
```

---

### Median

```
import statistics

nums = [10, 20, 30, 40]

print(statistics.median(nums))
```

Output:

```
25.0
```

---

## Mode

```
import statistics

nums = [1, 2, 2, 3]

print(statistics.mode(nums))
```

Output:

```
2
```

---

## 8. collections Module

Provides specialized container data types.

```
import collections
```

### Counter

Counts occurrences.

```
from collections import Counter

text = "banana"

print(Counter(text))
```

Output:

```
Counter({'a': 3, 'n': 2, 'b': 1})
```

---

### defaultdict

Provides default values.

```
from collections import defaultdict

d = defaultdict(int)

d["a"] += 1

print(d)
```

Output:

```
defaultdict(<class 'int'>, {'a': 1})
```

---

## deque

Fast insertion and deletion from both ends.

```
from collections import deque

dq = deque([1, 2, 3])

dq.appendleft(0)
dq.append(4)

print(dq)
```

Output:

```
deque([0, 1, 2, 3, 4])
```

---

## 9. `time` Module

Used for time-related operations.

```
import time
```

### Current Timestamp

```
import time

print(time.time())
```

Example Output:

```
1781002500.52
```

---

### Pause Execution

```
import time

print("Start")

time.sleep(3)

print("End")
```

Output:

```
Start
(waits 3 seconds)
End
```

---

# 10. `re` Module (Regular Expressions)

Used for pattern matching.

```
import re
```

## Find Digits

```
import re

text = "My number is 98765"

result = re.findall(r"\d+", text)

print(result)
```

Output:

```
['98765']
```

---

## Check Email Pattern

```
import re

email = "abc@gmail.com"

if re.match(r'^\w+@\w+\.\w+$', email):
    print("Valid")
else:
    print("Invalid")
```

Output:

```
Valid
```

---

# Summary Table

| Module                   | Purpose                        |
|--------------------------|--------------------------------|
| <code>math</code>        | Mathematical calculations      |
| <code>random</code>      | Random numbers and selections  |
| <code>datetime</code>    | Dates and times                |
| <code>os</code>          | Operating system operations    |
| <code>sys</code>         | Python interpreter information |
| <code>json</code>        | JSON data handling             |
| <code>statistics</code>  | Statistical calculations       |
| <code>collections</code> | Specialized data structures    |

| <b>Module</b> | <b>Purpose</b>                             |
|---------------|--------------------------------------------|
| time          | Time-related functions                     |
| re            | Pattern matching using regular expressions |

**Link :** [CHATGPT](#)

**Link :** [ALL](#)